UNIVERSITY OF RWANDA

CST

COMPUTER ENGINEERING

LEVEL IV

GAKO CAMPUS

REAL TIME SYSTEMS

Submission date: [27/10/2026]

GITHUB ACCOUNT: https://github.com/IRAFASHA-Justine/RT_Mini_Project_3_Report

REAL TIME SYSTEM MINI PROJECT

Unit 3: Concurrency, Synchronization, Deadlock and Interrupt Latency

---

GROUP 4_MEMBERS:

[Abdul SIBO] - [223026681]

[Justine IRAFASHA] - [223026693]

[Fils NIYONZIMA] - [223026876]

Table of Contents

Mini Project 3.1: Simulating RTOS Concurrency, Synchronization and Race Conditions 3

Race condition and its fix (Steps 1-4) 3
Bounded queue as an RTOS message queue (Steps 5-6) 3
Counting semaphore as a resource pool (Steps 7-8) 4
Dispatch latency (Step 9) 4
Answers to Lab Tasks 5
Conclusion: Project 3.1 6

Mini Project 3.2: Bounded-Buffer Producer-Consumer Simulator with Deadlock and Lock-Ordering Fix 7

Reproducing the deadlock 7
Coffman conditions mapped to the code 7
Lock-ordering fix 7
Throughput versus buffer capacity 8
Discussion and report: Project 3.2 8

Mini Project 3.3: Priority-Based Interrupt Latency Simulator 10

Results for three configurations 10
Arrival-rate sweep 11
Discussion and report: Project 3.3 12
Conclusion: Project 3.3 12

Appendix: Source Code 13

A. Mini Project 3.1 13
B. Mini Project 3.2 16
C. Mini Project 3.3 18

Mini Project 3.1: Simulating RTOS Concurrency, Synchronization and Race Conditions

Setup. Each Python thread stands in for one RTOS task. All scripts use only the standard library (threading, queue, time) and were run on Python 3.12.3 on a single-core Linux machine. Python's Global Interpreter Lock prevents true parallelism, which suits this exercise: it isolates interleaving hazards (Section 3.8) without the extra complexity of multi-core execution. threading.Lock plays the role of a mutex, threading.Semaphore a counting semaphore, and a bounded queue.Queue an RTOS message queue (Section 3.4). Timing uses time.perf_counter().

Race condition and its fix (Steps 1-4)

race_demo.py shares a global counter between five threads, each performing 1000 increments as a non-atomic read, a 0.1 ms sleep, then a write-back. The expected total is 5000. Three runs each of the unprotected and the mutex-protected versions gave:

```
run 1 (race_demo.py):  threads=5 expected=5000 actual=1000 lost=4000
run 2 (race_demo.py):  threads=5 expected=5000 actual=1000 lost=4000
run 3 (race_demo.py):  threads=5 expected=5000 actual=1000 lost=4000
run 1 (race_fixed.py): threads=5 expected=5000 actual=5000 lost=0
run 2 (race_fixed.py): threads=5 expected=5000 actual=5000 lost=0
run 3 (race_fixed.py): threads=5 expected=5000 actual=5000 lost=0
```

Observation. The unprotected version lost 4000 of 5000 updates in every run, not the run-to-run variation the manual predicts. The cause is that the deliberate sleep inside the read-modify-write window puts all five threads in lock-step: every thread reads the same value, all sleep, and all write back value + 1, so a whole round of five increments is counted as one. The result is therefore exactly 1000. This is a lost-update race (Section 3.8) in its most extreme, systematic form. Wrapping the critical section in with counter_lock: serialises the read-modify-write so no thread can read a stale value, and the result is exactly 5000 in every run.

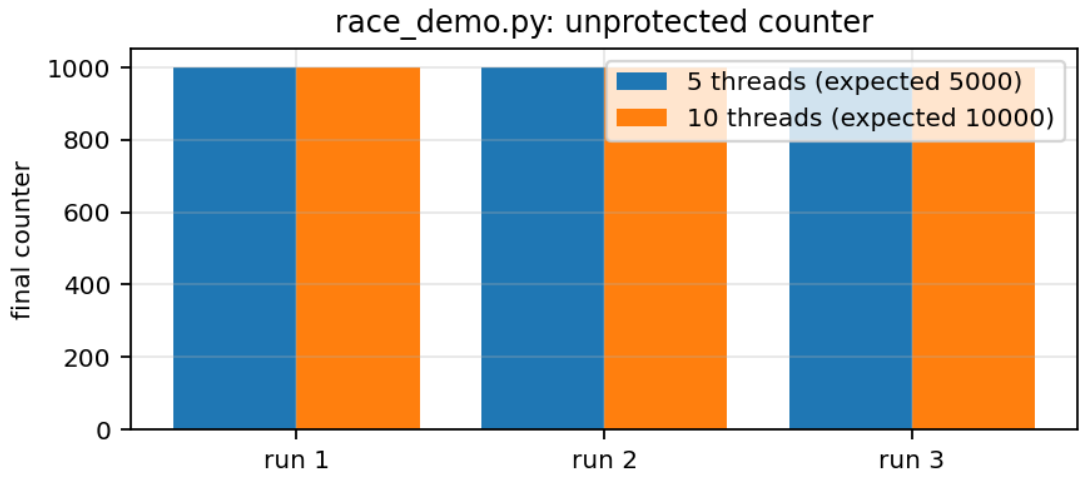


Figure 1. Final counter value with no lock, for 5 threads and 10 threads (three runs each).

Bounded queue as an RTOS message queue (Steps 5-6)

A producer put 20 timestamped items into queue.Queue(maxsize=5) and a consumer computed per-item latency from send to receipt. With a fast consumer, latency stayed at microsecond scale; adding a 5 ms delay to the consumer loop changed the picture sharply.

Consumer delay	Min (ms)	Max (ms)	Average (ms)	Producer puts that blocked	Total producer block (ms)
0 ms	0.006	0.058	0.022	0	0.1
5 ms	0.032	31.0	25.4	14	71.8

Slowing the consumer raised average latency from 0.022 ms to 25.4 ms, and the producer was forced to block on 14 of 20 puts. Once the queue is full, items wait behind four or five others, which is the response-latency growth discussed in Section 3.5.

Counting semaphore as a resource pool (Steps 7-8)

Eight task threads competed for Semaphore(3), each holding the resource for a random 50-150 ms. The peak number of simultaneous holders was 3, three tasks acquired immediately, and the other five waited (average wait 86.3 ms, worst 226.3 ms), which is exactly counting-semaphore blocking.

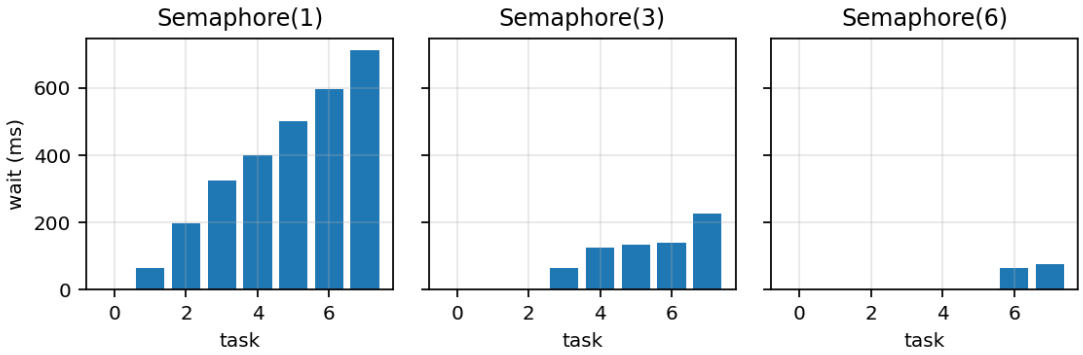


Figure 2. Per-task wait time before acquiring the resource for semaphore counts 1, 3 and 6 (Lab Task 2).

Dispatch latency (Step 9)

In 200 iterations a new thread set an Event and the main thread measured the time until event.wait() returned. Results: minimum 14.3 µs, mean 17.7 µs, 95th percentile 21.9 µs, and maximum 85.7 µs, the worst-case observed dispatch latency. The maximum is about 4.8 times the mean. That gap is why RTOS designers reason about the worst case rather than the average (Section 3.1): a system sized for the mean would be wrong on the rare occasions that matter.

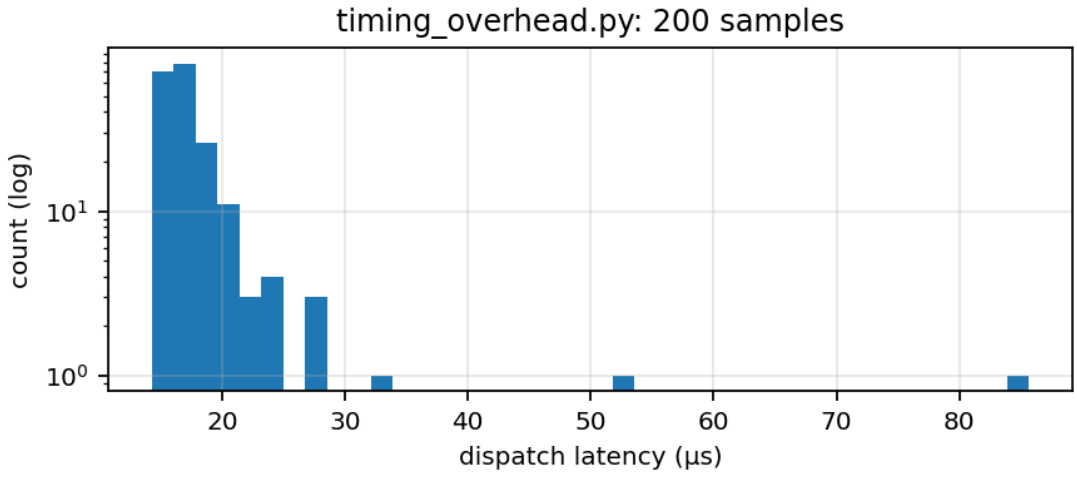


Figure 3. Distribution of 200 thread-dispatch latencies (log count axis); the long right tail is the worst case.

Answers to Lab Tasks

Lab Task 1: ten threads instead of five. With ten threads the final counter was 1002, 1001, 1001 against an expected 10000 (three runs), so about 9000 updates were lost, versus 4000 of 5000 with five threads. The absolute loss more than doubles and the lost fraction rises from 80% to about 90%. More tasks contending without protection make the lock-step collapse worse, because the counter advances by roughly one per round no matter how many threads take part.

Lab Task 2: semaphore count 1 and 6. With a count of 1 the pool behaves like a mutex: tasks run strictly one at a time, only 1 task acquires immediately, and the average wait rises to 349.5 ms (worst 711.4 ms). With a count of 3 the average wait is 86.3 ms. With a count of 6 there are six immediate acquisitions and only two tasks wait, giving an average of 17.4 ms (worst 75.6 ms). Peak holders always equalled the semaphore count. A larger count means a larger pool of identical resource blocks, so less contention and shorter blocked time, as Section 3.4 describes for counting semaphores.

Lab Task 3: livelock. Two threads each took one of two locks in opposite orders, tried for the second lock without blocking, and on failure released the first and retried in lock-step (a barrier keeps their attempts synchronized). In one second the retry counters reached 54,365 and 54,365 while completed work was 0 for both. Both threads stay active, burning CPU, but neither makes forward progress. That is the Section 3.8 definition of livelock: the tasks are not blocked as in deadlock, they keep changing state without getting anywhere. Breaking the symmetry, for example with a random back-off or a fixed lock order, resolves it.

Lab Task 4: queue maxsize sweep. With a 2 ms consumer delay:

maxsize	Average latency (ms)	Producer puts that blocked (of 20)
1	3.95	18
2	5.75	17
5	10.52	14
10	16.24	9
20	20.16	0

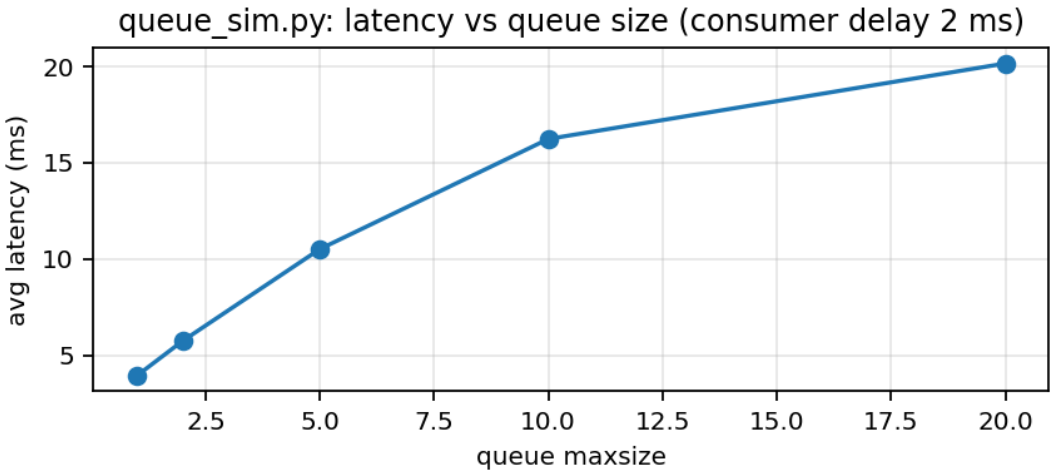


Figure 4. Average item latency versus queue size.

Average latency rises with queue size (3.95 ms at size 1 to 20.16 ms at size 20), while producer blocking falls to zero once the queue can hold all 20 items. A deeper queue absorbs bursts and keeps the producer

running, but items sit longer waiting for the slow consumer. A designer must therefore size RTOS queues to the burst length that must be absorbed, not simply make them as large as memory allows.

Lab Task 5: Semaphore versus Lock and priority inheritance. Section 3.4 presents the mutex as an ownership-aware primitive: it records which task holds it, and only that task may release it. Priority inheritance depends on that record, because when a high-priority task blocks on the mutex the kernel must know which low-priority holder to boost temporarily. A counting semaphore has no owner. Any task may release it, and it only tracks a count, so the kernel has no holder to promote, and a medium-priority task can preempt the resource holder indefinitely (priority inversion). A plain threading.Semaphore would therefore be unsuitable wherever resource-ownership reasoning is required. (In Python, threading.Lock can technically be released by a different thread, and RLock is the owner-tracking variant, which shows that the ownership semantics of the RTOS mutex are a design property, not something every library lock guarantees.)

Conclusion: Project 3.1

This project reproduced the Unit 3 hazards in a controlled sandbox. A lost-update race was produced and eliminated with a mutex; a bounded queue and a counting semaphore showed how blocking converts shared-resource contention into measurable latency; and the dispatch-latency histogram showed that the worst case sits far from the mean. The livelock experiment also showed that a system can be fully busy and make no progress at all.

Mini Project 3.2: Bounded-Buffer Producer-Consumer Simulator with Deadlock and Lock-Ordering Fix

Setup. Three producer and three consumer threads share a bounded buffer (a Python list guarded by buffer_lock) and a statistics dictionary (guarded by stats_lock). Explicit locks are used instead of queue.Queue so that acquisition order is fully controlled. Producers and consumers take a random 0.5-5 ms to make or use an item, and a full or empty buffer causes a short back-off and retry. Capacity, task counts and item totals are function parameters. A watchdog thread reports a deadlock when the produced-item count has not changed for 1 second.

Reproducing the deadlock

In the flawed version producers take buffer_lock then stats_lock, while consumers take stats_lock then buffer_lock. A short sleep between the two acquisitions widens the window. The watchdog fired in all 5 of 5 runs (seeds 0 to 4), each time after exactly one second of no progress. Evidence from one run:

```
[WATCHDOG] DEADLOCK: produced count frozen at 0 for 1.0 s (consumed=0)
   C1: holds nothing, WAITING for stats_lock
   C2: holds stats_lock, WAITING for buffer_lock
   C3: holds nothing, WAITING for stats_lock
   P1: holds nothing, WAITING for buffer_lock
   P2: holds buffer_lock, WAITING for stats_lock
   P3: holds nothing, WAITING for buffer_lock
```

P3 holds buffer_lock and waits for stats_lock, while C3 holds stats_lock and waits for buffer_lock. Neither can proceed and all other tasks queue behind them. In the run shown, no item was ever produced before the system froze.

Coffman conditions mapped to the code

Coffman condition	Where it holds in the flawed code
Mutual exclusion	buffer_lock and stats_lock are threading.Lock objects: only one thread can hold each at a time.
Hold and wait	The producer keeps buffer_lock while it requests stats_lock; the consumer keeps stats_lock while it requests buffer_lock.
No preemption	A Python Lock cannot be taken away from its holder; it is released only by the holder itself.
Circular wait	Producers acquire buffer then stats, consumers acquire stats then buffer, so P waits on C and C waits on P.

Lock-ordering fix

In the corrected version every task acquires buffer_lock first and stats_lock second. With a single global order, a cycle in the wait-for graph is impossible, so **circular wait** is eliminated, and since all four conditions are required, deadlock cannot occur. The other three conditions remain true; only the fourth was removed. The fixed version completed all 600 items in about 0.6 s (996 items/s), and the watchdog never fired.

```
completed in 0.60 s, throughput=996 items/s
```

Throughput versus buffer capacity

The corrected simulation was run with 3 producers, 3 consumers and 600 items at six capacities, averaging three runs per point.

Capacity	Throughput (items/s)	Avg producer block (ms/item)	Avg consumer block (ms/item)
1	896	0.370	0.357
2	924	0.267	0.251
5	954	0.118	0.132
10	964	0.050	0.107
25	985	0.002	0.059
50	985	0.002	0.066

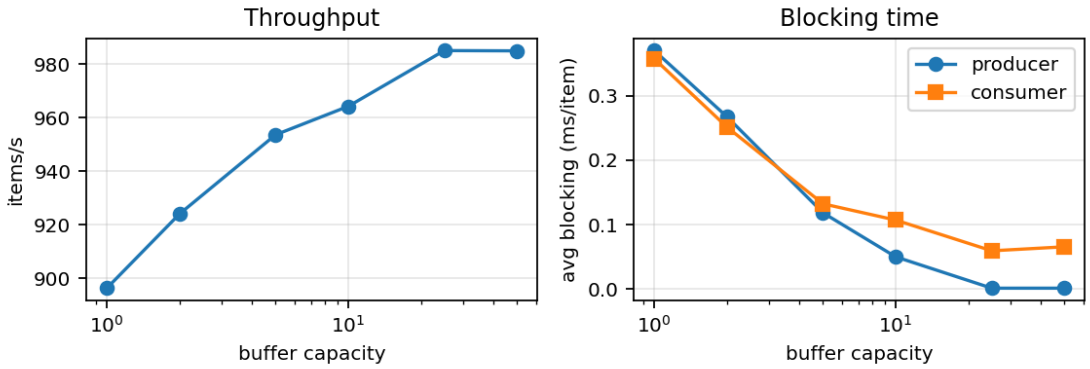


Figure 5. Throughput and average blocking time as a function of buffer capacity (log scale on x).

Discussion and report: Project 3.2

The flawed design satisfies all four of Coffman's necessary conditions (Section 3.8) at once, and the experiment shows that this is not merely theoretical. Each condition is visible in a specific line of code, listed in the table above. The deadlock appeared in every run because the two critical sections were deliberately long enough for the opposite lock orders to interleave. In a real system the window could be microseconds wide, and the failure would appear only rarely under load, which is the dangerous property of deadlock: it is nondeterministic in testing but catastrophic in the field. The watchdog report is therefore as much part of the engineering result as the fix, because it turned a silent freeze into a diagnosable event listing which thread held which lock.

The fix targets only circular wait. Mutual exclusion cannot be removed here, because the buffer and the statistics are genuinely shared mutable state. Eliminating hold-and-wait would mean acquiring both locks atomically, or releasing one before taking the other, which would complicate the code and allow the statistics to disagree with the buffer momentarily. Preemption of locks is not available in Python and is generally unsafe because the protected data may be half-updated. A global lock order is the cheapest approach: it costs nothing at runtime and only requires a documented convention that every task follows. Its weakness is that it relies on programmer discipline. If a single new function took the locks in the opposite order, the deadlock would return, which is why the order should be recorded in comments and enforced in code review.

The throughput results show the expected shape. With capacity 1, producers and consumers must nearly alternate, so any randomness in service times causes one side to wait for the other: throughput was 896

items/s and the producer waited 0.37 ms per item. Larger buffers absorb the random variation in how long each item takes to produce or consume. Throughput rose to 954 items/s at capacity 5 and 985 items/s at capacity 25, roughly a 10% improvement over capacity 1, while producer blocking fell from 0.37 ms to 0.0016 ms per item. From capacity 25 to 50, throughput did not change (985 versus 985 items/s) and the blocking times barely moved. Capacity of about 25 is therefore the point of diminishing returns for this workload.

The plateau exists because the system is then limited by the service rate of the consumers, not by the buffer. Three consumers averaging about 2.75 ms per item can handle at most roughly 1,090 items/s, and the measured plateau of about 985 items/s is close to that ceiling once lock and back-off overhead are included. Once the buffer is deep enough to hide short bursts, extra capacity only stores items that were never going to be processed any faster. In a real-time system there is also a cost to over-sizing: memory is consumed and the worst-case waiting time for an item inside the buffer grows, as Lab Task 4 of Project 3.1 showed. The recommended design therefore sizes the buffer to the burst length it must absorb, which here is a few tens of items, rather than choosing the largest possible value.

In summary, the project demonstrates the full cycle: a design that satisfies all four Coffman conditions deadlocks under load; a watchdog makes the failure observable; a consistent lock order removes circular wait and therefore deadlock; and buffer capacity is a tuning parameter with a clear knee in the curve rather than a free resource.

Mini Project 3.3: Priority-Based Interrupt Latency Simulator

Setup. Three periodic interrupt sources (A, B, C; priority 1 is highest) are simulated with real wall-clock time scaled to milliseconds. Each source is a thread that sleeps until its next absolute assertion time, then runs its **top half** while holding a global interrupt lock, so a top half occupies the simulated CPU and preempts any bottom half. At the end of the top half it queues a **bottom half** in a priority heap (heapq) tagged with its priority and assertion timestamp. One CPU worker runs the highest-priority bottom half in 1 ms slices and re-queues the remainder, so a newly arrived higher-priority bottom half preempts a running one mid-execution, and the worker pauses while any top half runs. Interrupt latency is measured from the scheduled assertion to the first instruction of the top half, and response latency from assertion to completion of the bottom half (Section 3.5).

Source	Priority	Period (ms)	Top half (ms)	Bottom half (ms)
A (hi)	1	20	0.5	3
B (mid)	2	50	0.5	8
C (lo)	3	100	0.5	15

Each configuration was run for 6 seconds (the manual suggests about 20; the duration is a parameter and was shortened so the full set of runs fit in the available session).

Results for three configurations

Configuration 1 baseline (short top halves)

Source	Events	Min (ms)	Mean (ms)	Max (ms)	p95 (ms)	Max interrupt latency (ms)
A (hi)	297	3.9	4.6	6.6	6.3	1.5
B (mid)	119	9.3	12.0	15.3	15.0	1.4
C (lo)	59	32.5	33.7	35.0	34.0	1.4

Configuration 2 long top half on B

Source	Events	Min (ms)	Mean (ms)	Max (ms)	p95 (ms)	Max interrupt latency (ms)
A (hi)	297	3.9	7.6	20.1	17.9	13.0
B (mid)	119	20.8	27.4	37.2	30.6	1.4
C (lo)	59	47.1	49.0	64.0	49.6	12.8

Configuration 3 heavy load (A period 10 ms)

Source	Events	Min (ms)	Mean (ms)	Max (ms)	p95 (ms)	Max interrupt latency (ms)
A (hi)	595	3.7	4.7	8.1	6.2	3.5
B (mid)	119	13.2	17.5	28.9	18.5	2.5
C (lo)	59	40.3	40.9	50.6	41.3	1.8

Preemption evidence. The timeline below shows a 200 ms window of the baseline run. Red bars are top halves, which cut into whatever the CPU was doing, and the coloured bars are 1 ms bottom-half slices. Source C's long bottom half (15 ms) is repeatedly interrupted by the arrival of A's and B's work and is resumed only when nothing of higher priority is ready.

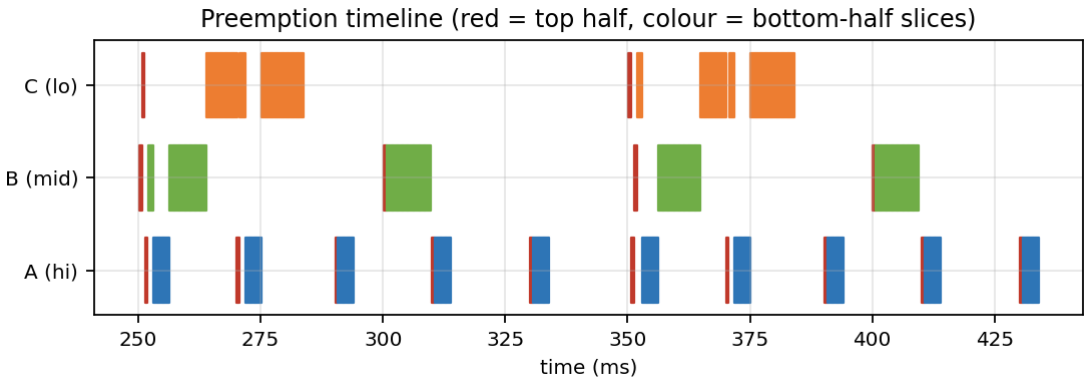


Figure 6. Preemption timeline for a representative 200 ms window of the baseline configuration.

Arrival-rate sweep

The period of the highest-priority source A was varied while the other sources were left unchanged (4 seconds per point). Source C has the lowest priority, so it is the most exposed.

A period (ms)	C mean (ms)	C p95 (ms)	C max (ms)
40	29.9	30.7	31.5
20	34.8	34.4	69.0
10	40.8	41.7	48.4
7	70.6	76.3	77.3

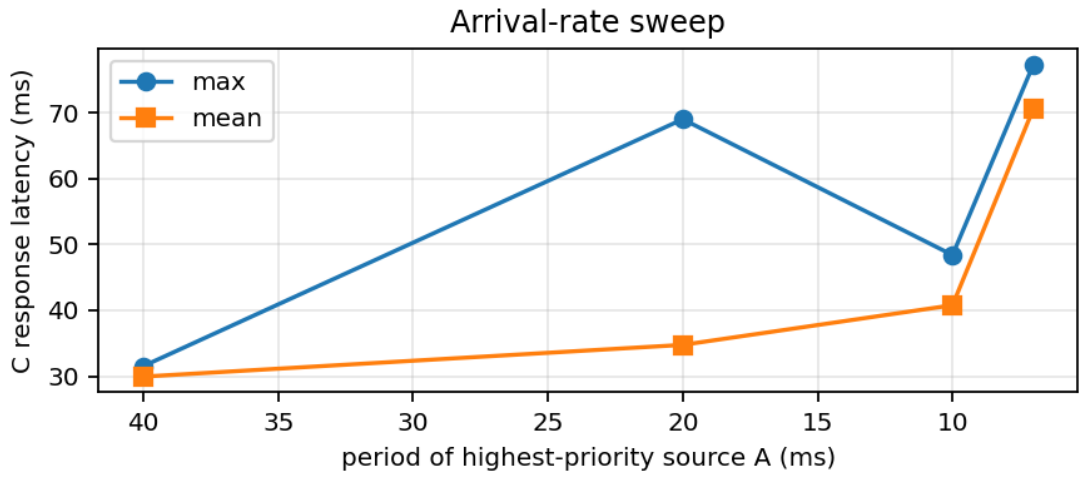


Figure 7. Response latency of the lowest-priority source C as the highest-priority source A fires more often.

Caveat on the maximum. Mean and 95th percentile rise steadily with arrival rate, but the maximum is noisy: at A period 20 ms (the same setting as the baseline) the sweep recorded a maximum of 69.0 ms against 35.0 ms in the baseline run. A maximum is a single worst sample, and on a shared machine with sleep-based timing one scheduling hiccup of the host operating system can dominate it. This reinforces the point of Project 3.1: a measured maximum is an observation, not a proven bound.

Discussion and report: Project 3.3

Section 3.5 separates the interrupt handler into a short top half, which acknowledges the hardware and queues work, and a bottom half, which is an ordinary schedulable task. The reason is that top halves run above every task priority: nothing, however important, can run while one is executing. In the baseline configuration all three top halves take 0.5 ms, so the interrupt latency of every source stayed small (mean 0.3 to 0.5 ms, worst 1.5 ms) and the response latency of each source was dominated by its own bottom-half length and by higher-priority bottom halves, which is the intended, analysable behaviour. Source A, the highest priority, responded in 4.6 ms on average for 3 ms of work, source B in 12.0 ms for 8 ms, and source C in 33.7 ms for 15 ms. Latency increased with each step down in priority, as strict priority scheduling predicts.

The second configuration breaks the guideline by giving source B a 12 ms top half. The effect is much wider than B itself. Source A has the highest priority, and neither its period nor its own durations changed, yet its worst response latency rose from 6.6 ms to 20.1 ms (about 3.0 times) and its worst interrupt latency from 1.5 ms to 13.0 ms. This is because A's interrupt arrived while B's top half held the CPU and had to wait for it to finish. Priority could not help, since the long top half runs at interrupt level, above all priorities. Source C was affected too: its mean response latency went from 33.7 ms to 49.0 ms and its worst case reached 64.0 ms. B itself had a mean response of 27.4 ms against 12.0 ms before. The lesson from Section 3.5 is confirmed directly: the cost of a long top half is paid by every other source in the system, and the worst-case interrupt latency of any source is bounded below by the longest top half anywhere.

The third configuration, with A firing every 10 ms instead of 20 ms, shows the effect of arrival rate. A's own response did not change (mean 4.7 ms), because nothing outranks its bottom half apart from the brief top halves. B and C suffered: C's mean response rose from 33.7 ms to 40.9 ms and B's from 12.0 ms to 17.5 ms. Each extra arrival of A preempts them and adds 3.5 ms of work (top plus bottom half) to the time they need to finish. This is the same effect as a longer top half, but it comes through load rather than through a single long blocking interval.

The sweep makes the trend explicit. As the period of A fell from 40 ms to 7 ms, C's mean response latency rose from 29.9 ms to 70.6 ms and its 95th percentile from 30.7 ms to 76.3 ms. The increase is gentle at first and then steep, because the total load approaches the capacity of the one simulated CPU. At the shortest period the combined top-half and bottom-half demand is roughly 80% of the CPU, and C, whose period (and implicit deadline) is 100 ms, needed about 70 ms on average to complete, leaving little margin. Had the period been shortened a little more, C would have missed its deadline altogether.

These results connect to schedulability. A schedulability guarantee for a task assumes that the interference it suffers from higher-priority work is bounded and known. Top halves are the highest-priority work of all, so any uncertainty in their duration or arrival rate is inherited by every task beneath them. A long or unbounded top half, or an interrupt storm, makes the interference term in every lower task's response-time analysis unbounded, and no guarantee for the lower tasks can then be given. The practical rules from Section 3.5 follow: keep top halves short and of bounded duration, move everything else into prioritised bottom-half tasks that can be analysed with the normal scheduling theory, and, where an interrupt source can fire at an uncontrolled rate, limit its minimum inter-arrival time so that its interference stays bounded.

Conclusion: Project 3.3

The simulator reproduced the top-half/bottom-half pattern, showed bottom-half preemption mid-execution, and quantified how one long top half or a faster high-priority source degrades everything

beneath it. Mean and 95th-percentile latency gave stable trends, while the single-sample maximum was noisy, a reminder that real worst-case guarantees come from analysis and not from measurement alone.

Appendix: Source Code

Complete source of every script used to produce the results in this report.

A. Mini Project 3.1

race_demo.py

```
"""Mini Project 3.1 - Steps 1-3: lost-update race condition (Section 3.8)."""
import threading
import time
import sys

shared_counter = 0  # shared state, deliberately unprotected

def increment_unsafe(iterations):
    global shared_counter
    for _ in range(iterations):
        value = shared_counter           # 1. read
        time.sleep(0.0001)               # widen the interleaving window
        shared_counter = value + 1       # 2. write back (stale value possible)

def run(num_threads=5, iterations=1000):
    global shared_counter
    shared_counter = 0
    threads = [threading.Thread(target=increment_unsafe, args=(iterations,))
               for _ in range(num_threads)]
    for t in threads:
        t.start()
    for t in threads:
        t.join()
    expected = num_threads * iterations
    lost = expected - shared_counter
    print(f"threads={num_threads} expected={expected} actual={shared_counter} lost={lost}")

if __name__ == "__main__":
    n = int(sys.argv[1]) if len(sys.argv) > 1 else 5   # Lab Task 1: pass 10
    run(num_threads=n)
```

race_fixed.py

```
"""Mini Project 3.1 - Step 4: race fixed with a mutex (Section 3.8)."""
import threading
import time
import sys

shared_counter = 0
counter_lock = threading.Lock()  # mutex guarding the critical section

def increment_safe(iterations):
    global shared_counter
    for _ in range(iterations):
        with counter_lock:               # critical section: read-modify-write is now atomic
            value = shared_counter
            time.sleep(0.0001)
            shared_counter = value + 1

def run(num_threads=5, iterations=1000):
    global shared_counter
    shared_counter = 0
    threads = [threading.Thread(target=increment_safe, args=(iterations,))
               for _ in range(num_threads)]
    for t in threads:
        t.start()
    for t in threads:
        t.join()
    expected = num_threads * iterations
    lost = expected - shared_counter
    print(f"threads={num_threads} expected={expected} actual={shared_counter} lost={lost}")

if __name__ == "__main__":
    n = int(sys.argv[1]) if len(sys.argv) > 1 else 5   # Lab Task 1: pass 10
    run(num_threads=n)
```

queue_sim.py

```
"""Mini Project 3.1 - Steps 5-6: bounded queue as an RTOS message queue (Section 3.4)."""
import queue, threading, time, json, sys

ITEMS = 20

def run(maxsize=5, consumer_delay=0.0):
    q = queue.Queue(maxsize=maxsize)
    latencies, block_times = [], []

    def producer_task():
        for i in range(ITEMS):
            t0 = time.perf_counter()
            q.put((i, time.perf_counter()))          # blocks while the queue is full
            block_times.append(time.perf_counter() - t0)

    def consumer_task():
        for _ in range(ITEMS):
            i, sent = q.get()
            latencies.append(time.perf_counter() - sent)   # send -> receipt
            if consumer_delay:
                time.sleep(consumer_delay)                 # slow consumer

    p = threading.Thread(target=producer_task); c = threading.Thread(target=consumer_task)
    p.start(); c.start(); p.join(); c.join()
    blocked = sum(1 for b in block_times if b > 0.001)
    return {"maxsize": maxsize, "consumer_delay_ms": consumer_delay * 1e3,
            "min_ms": min(latencies) * 1e3, "max_ms": max(latencies) * 1e3,
            "avg_ms": sum(latencies) / len(latencies) * 1e3,
            "producer_blocked_puts": blocked, "producer_block_total_ms": sum(block_times) * 1e3}

if __name__ == "__main__":
    for d in (0.0, 0.005):
        r = run(5, d)
        print(f"consumer_delay={r['consumer_delay_ms']:.0f} ms  min={r['min_ms']:.3f}  max={r['max_ms']:.3f}
"
              f"avg={r['avg_ms']:.3f} ms  blocked_puts={r['producer_blocked_puts']}  "
              f"total_block={r['producer_block_total_ms']:.1f} ms")
```

semaphore_pool.py

```
"""Mini Project 3.1 - Steps 7-8: counting semaphore as a pool of fixed-size blocks."""
import threading, time, random

def run(count=3, tasks=8, seed=1):
    random.seed(seed)
    sem = threading.Semaphore(count)
    state = threading.Lock(); holding = 0; peak = 0
    waits = {}
    holds = [random.uniform(0.05, 0.15) for _ in range(tasks)]

    def task(tid):
        nonlocal holding, peak
        t0 = time.perf_counter()
        sem.acquire()
        waits[tid] = time.perf_counter() - t0
        with state:
            holding += 1; peak = max(peak, holding)
        time.sleep(holds[tid])                    # use the resource
        with state:
            holding -= 1
        sem.release()

    ts = [threading.Thread(target=task, args=(i,)) for i in range(tasks)]
    for t in ts: t.start()
    for t in ts: t.join()
    w = [waits[i] * 1e3 for i in range(tasks)]
    return {"count": count, "waits_ms": w, "peak_holders": peak,
            "immediate": sum(1 for x in w if x < 5), "avg_wait_ms": sum(w) / len(w), "max_wait_ms": max(w)}

if __name__ == "__main__":
    r = run(3)
    for i, w in enumerate(r["waits_ms"]): print(f"task {i}: wait = {w:7.2f} ms")
    print(f"peak simultaneous holders = {r['peak_holders']}, immediate = {r['immediate']}, "
          f"avg wait = {r['avg_wait_ms']:.1f} ms")
```

timing_overhead.py

```
"""Mini Project 3.1 - Step 9: thread-dispatch latency (analogy to Section 3.5)."""
import threading, time, statistics, json

def measure(n=200):
    out = []
    for _ in range(n):
        ev = threading.Event()
        t = threading.Thread(target=ev.set)
        t0 = time.perf_counter()
        t.start()
        ev.wait()
        out.append((time.perf_counter() - t0) * 1e6)   # microseconds
        t.join()
    return out

if __name__ == "__main__":
    x = measure()
    s = sorted(x)
    print(f"n={len(x)}  min={s[0]:.1f} us  mean={statistics.mean(x):.1f} us  "
          f"p95={s[int(0.95 * len(s)) - 1]:.1f} us  max={s[-1]:.1f} us (worst-case observed dispatch
latency)")
```

livelock_demo.py

```
"""Mini Project 3.1 - Lab Task 3: livelock with two locks, opposite order, back-off and retry."""
import threading, time

def run(duration=1.0):
    A, B = threading.Lock(), threading.Lock()
    bar = threading.Barrier(2)
    retries = [0, 0]; progress = [0, 0]; stop = time.perf_counter() + duration

    def worker(idx, first, second):
        try:
            while time.perf_counter() < stop:
                first.acquire()
                bar.wait(timeout=1)                    # both hold their first lock
                got = second.acquire(blocking=False)   # fails: the peer holds it
                bar.wait(timeout=1)                    # both have attempted
                if got:
                    progress[idx] += 1; second.release(); first.release(); return
                first.release()                        # back off ...
                retries[idx] += 1                      # ... and retry in lock-step
                bar.wait(timeout=1)
        except threading.BrokenBarrierError:
            pass

    t1 = threading.Thread(target=worker, args=(0, A, B)); t2 = threading.Thread(target=worker, args=(1, B,
A))
    t1.start(); t2.start(); t1.join(); t2.join()
    return retries, progress

if __name__ == "__main__":
    r, p = run()
    print(f"retries T1={r[0]} T2={r[1]}  completed work T1={p[0]} T2={p[1]}")
```

lab_tasks.py

```
"""Mini Project 3.1 - Lab Tasks driver: collects all measurements into results/mp31.json."""
import json, statistics, io, contextlib
import race_demo, queue_sim, semaphore_pool, timing_overhead, livelock_demo

res = {}
# Race demo: 3 runs x (5 and 10 threads)
def race(n, runs=3):
    out = []
    for _ in range(runs):
        buf = io.StringIO()
        with contextlib.redirect_stdout(buf): race_demo.run(num_threads=n)
        out.append(race_demo.shared_counter)
    return out
res["race5"] = race(5); res["race10"] = race(10)
# Queue
res["queue_fast"] = queue_sim.run(5, 0.0); res["queue_slow"] = queue_sim.run(5, 0.005)
res["queue_sweep"] = [queue_sim.run(m, 0.002) for m in (1, 2, 5, 10, 20)]
# Semaphore
res["sem"] = {c: semaphore_pool.run(c) for c in (1, 3, 6)}
# Timing
```

```
x = timing_overhead.measure(200); s = sorted(x)
res["timing"] = {"samples": x, "min": s[0], "mean": statistics.mean(x), "p95": s[189], "max": s[-1]}
# Livelock
r, p = livelock_demo.run(1.0); res["livelock"] = {"retries": r, "progress": p}
json.dump(res, open("results/mp31.json", "w"))
print("done")
```

B. Mini Project 3.2

bounded_buffer.py

```
"""Mini Project 3.2: bounded-buffer producer/consumer with deadlock reproduction and lock-ordering fix.
Flawed:  producers take buffer_lock -> stats_lock, consumers take stats_lock -> buffer_lock (circular wait).
Fixed:   every task takes buffer_lock -> stats_lock (global lock order)."""
import threading, time, random, sys, json


def simulate(flawed, capacity=5, n_prod=3, n_cons=3, total=600, watchdog_s=1.0, seed=7, verbose=False):
    random.seed(seed)
    buffer, buffer_lock, stats_lock = [], threading.Lock(), threading.Lock()
    stats = {"produced": 0, "consumed": 0, "prod_wait": 0.0, "cons_wait": 0.0}
    state = {}                                  # thread name -> "holds X / waiting for Y"
    done = threading.Event(); deadlock = {}
    per = total // n_prod

    def acquire(name, lock, lname, held):
        state[name] = f"holds {held or 'nothing'}, WAITING for {lname}"
        lock.acquire(); state[name] = f"holds {(held + ' + ' if held else '') + lname}"


    def producer(name):
        for _ in range(per):
            time.sleep(random.uniform(0.0005, 0.005))       # produce the item
            t0 = time.perf_counter()
            while True:
                if flawed:
                    acquire(name, buffer_lock, "buffer_lock", "")
                    if len(buffer) < capacity:
                        buffer.append(1); time.sleep(0.001)  # widen the window
                        acquire(name, stats_lock, "stats_lock", "buffer_lock")
                        stats["produced"] += 1; stats["prod_wait"] += time.perf_counter() - t0
                        stats_lock.release(); buffer_lock.release(); break
                    buffer_lock.release()
                else:
                    with buffer_lock:
                        if len(buffer) < capacity:
                            buffer.append(1)
                            with stats_lock:
                                stats["produced"] += 1; stats["prod_wait"] += time.perf_counter() - t0
                            break
                time.sleep(0.0003)                          # buffer full: back off and retry
        state[name] = "finished"

    def consumer(name):
        for _ in range(total // n_cons):
            t0 = time.perf_counter()
            while True:
                if flawed:
                    acquire(name, stats_lock, "stats_lock", "")
                    time.sleep(0.001)                       # widen the window
                    acquire(name, buffer_lock, "buffer_lock", "stats_lock")
                    if buffer:
                        buffer.pop(); stats["consumed"] += 1; stats["cons_wait"] += time.perf_counter() - t0
                        buffer_lock.release(); stats_lock.release(); break
                    buffer_lock.release(); stats_lock.release()
                else:
                    with buffer_lock:
                        if buffer:
                            buffer.pop()
                            with stats_lock:
                                stats["consumed"] += 1; stats["cons_wait"] += time.perf_counter() - t0
                            break
                time.sleep(0.0003)
            time.sleep(random.uniform(0.0005, 0.005))       # consume the item
        state[name] = "finished"

    def watchdog():
        last, last_t = -1, time.perf_counter()
        while not done.is_set():
            time.sleep(0.1)
            if stats["produced"] != last:
```

```
                last, last_t = stats["produced"], time.perf_counter()
            elif time.perf_counter() - last_t > watchdog_s:
                deadlock.update(produced=stats["produced"], consumed=stats["consumed"],
snapshot=dict(state),
                                stalled_for=time.perf_counter() - last_t)
                done.set(); return

    threads = [threading.Thread(target=producer, args=(f"P{i+1}",), daemon=True) for i in range(n_prod)] + \
              [threading.Thread(target=consumer, args=(f"C{i+1}",), daemon=True) for i in range(n_cons)]
    wd = threading.Thread(target=watchdog, daemon=True)
    t_start = time.perf_counter(); wd.start()
    for t in threads: t.start()
    while not done.is_set() and any(t.is_alive() for t in threads): time.sleep(0.02)
    elapsed = time.perf_counter() - t_start
    if deadlock:
        return {"deadlock": True, **deadlock, "elapsed": elapsed}
    done.set()
    return {"deadlock": False, "elapsed": elapsed, "capacity": capacity, "throughput": total / elapsed,
            "avg_prod_block_ms": stats["prod_wait"] / total * 1e3, "avg_cons_block_ms": stats["cons_wait"] /
total * 1e3}

if __name__ == "__main__":
    flawed = "--flawed" in sys.argv
    r = simulate(flawed)
    if r["deadlock"]:
        print(f"[WATCHDOG] DEADLOCK: produced count frozen at {r['produced']} for {r['stalled_for']:.1f} s
(consumed={r['consumed']})")
        for k, v in sorted(r["snapshot"].items()): print(f"   {k}: {v}")
    else:
        print(f"completed in {r['elapsed']:.2f} s, throughput={r['throughput']:.0f} items/s")
```

bb_experiments.py

```
import json, bounded_buffer as bb
res = {"flawed": [bb.simulate(True, seed=s) for s in range(5)],
       "fixed": bb.simulate(False),
       "sweep": []}
for cap in (1, 2, 5, 10, 25, 50):
    runs = [bb.simulate(False, capacity=cap, seed=s) for s in range(3)]
    res["sweep"].append({k: sum(r[k] for r in runs) / 3 for k in ("throughput", "avg_prod_block_ms",
"avg_cons_block_ms")} | {"capacity": cap})
json.dump(res, open("results/mp32.json", "w"), default=str); print("ok")
```

C. Mini Project 3.3

irq_sim.py

```
"""Mini Project 3.3: priority-based interrupt latency simulator (top half / bottom half, Section 3.5).
Wall-clock simulation, times in ms. Top halves preempt everything (hold the 'CPU' via irq_lock);
one CPU worker runs bottom halves by strict priority in 1 ms slices, so a newly queued higher-priority
bottom half preempts a running one mid-execution (remainder is re-queued)."""
import threading, time, heapq, itertools, statistics, json

SLICE = 0.001

def run(sources, duration=6.0, record_window=None):
    """sources: list of dict(name, prio(1=highest), period_ms, top_ms, bottom_ms)"""
    irq_lock = threading.Lock()                 # top halves occupy the CPU one at a time
    cv = threading.Condition(); heap = []; cnt = itertools.count()
    stats = {s["name"]: {"irq_lat": [], "top": [], "resp": []} for s in sources}
    segs = []; preempts = [0]
    t0 = time.perf_counter(); stop = t0 + duration
    irq_active = [0]

    def source(s):
        k = 1
        while True:
            assertion = t0 + 0.05 + k * s["period_ms"] / 1e3; k += 1
            if assertion > stop: return
            d = assertion - time.perf_counter()
            if d > 0: time.sleep(d)
            with irq_lock:                                  # top half starts
                start = time.perf_counter(); irq_active[0] = 1
                time.sleep(s["top_ms"] / 1e3)               # top-half work
                end = time.perf_counter(); irq_active[0] = 0
            stats[s["name"]]["irq_lat"].append((start - assertion) * 1e3)
            stats[s["name"]]["top"].append((end - start) * 1e3)
            if record_window: segs.append((start - t0, end - t0, s["name"], "top"))
            with cv:                                        # defer the rest to a bottom half
                heapq.heappush(heap, [s["prio"], next(cnt), assertion, s["bottom_ms"] / 1e3, s["name"]])
```

```
                cv.notify()

    def cpu():
        last = None
        while True:
            with cv:
                while not heap:
                    if time.perf_counter() > stop + 0.5: return
                    cv.wait(0.05)
                task = heapq.heappop(heap)
            while irq_active[0]: time.sleep(0.0002)          # CPU busy with a top half
            if last is not None and last is not task and last[3] > 0: preempts[0] += 1
            st = time.perf_counter(); time.sleep(min(SLICE, task[3])); en = time.perf_counter()
            task[3] -= min(SLICE, task[3])
            if record_window: segs.append((st - t0, en - t0, task[4], "bottom"))
            if task[3] <= 1e-9:
                stats[task[4]]["resp"].append((en - task[2]) * 1e3); last = None
            else:
                with cv: heapq.heappush(heap, task)
                last = task

    ts = [threading.Thread(target=source, args=(s,)) for s in sources]; w = threading.Thread(target=cpu)
    for t in ts: t.start()
    w.start()
    for t in ts: t.join()
    w.join()
    out = {}
    for n, d in stats.items():
        r = sorted(d["resp"])
        out[n] = {"n": len(r), "min": r[0], "mean": statistics.mean(r), "max": r[-1], "p95": r[max(0,
int(0.95 * len(r)) - 1)],
                  "irq_lat_mean": statistics.mean(d["irq_lat"]), "irq_lat_max": max(d["irq_lat"]),
                  "top_mean": statistics.mean(d["top"])}
    return out, segs

BASE = [dict(name="A (hi)", prio=1, period_ms=20, top_ms=0.5, bottom_ms=3),
        dict(name="B (mid)", prio=2, period_ms=50, top_ms=0.5, bottom_ms=8),
        dict(name="C (lo)", prio=3, period_ms=100, top_ms=0.5, bottom_ms=15)]
def cfg(**over):
    return [dict(s, **over.get(s["name"][0], {})) for s in BASE]

if __name__ == "__main__":
    res = {}
    configs = {"1 baseline (short top halves)": cfg(),
               "2 long top half on B": cfg(B=dict(top_ms=12)),
               "3 heavy load (A period 10 ms)": cfg(A=dict(period_ms=10))}
    for name, c in configs.items():
        res[name], _ = run(c, 6.0)
        print(name, {k: round(v["max"], 1) for k, v in res[name].items()})
    sweep = {}
    for p in (40, 20, 10, 7):
        r, _ = run(cfg(A=dict(period_ms=p)), 4.0); sweep[p] = r
        print("sweep A period", p, "-> C max", round(r["C (lo)"]["max"], 1))
    _, segs = run(BASE, 1.0, record_window=True)
    json.dump({"configs": res, "sweep": sweep, "segs": [s for s in segs if 0.25 < s[0] < 0.45]},
open("results/mp33.json", "w"))
```